End-to-End Data Engineering Project Using PostgreSQL, Spark, MinIO, Airflow and Docker

Project Overview

This project demonstrates an end-to-end local Data Engineering pipeline using:

- PostgreSQL as source database
- Apache Spark for data transformation
- MinIO as local S3 storage
- Apache Airflow for orchestration
- Docker Compose for container management

The pipeline reads data from PostgreSQL, applies transformations using Spark, and writes parquet files into MinIO (S3-compatible object storage). Airflow orchestrates the entire workflow.

Final Architecture

```
PostgreSQL
     ↓
Spark ETL Job
     ↓
MinIO (S3)
     ↓
Airflow Orchestration
```

Tech Stack

Component	Purpose
PostgreSQL	Source database
Apache Spark	Distributed data processing
MinIO	Local S3-compatible object storage
Apache Airflow	Workflow orchestration
Docker Compose	Container management

Component	Purpose
PySpark	Spark processing using Python
Parquet	Optimized storage format

Project Folder Structure

```
DE_PROJECT/
│
├── airflow/
│   └── dags/
│       └── postgres_to_minio_dag.py
│
├── jars/
│   └── postgresql-42.7.11.jar
│
├── extra_jars/
│   ├── hadoop-aws-3.3.4.jar
│   └── aws-java-sdk-bundle-1.12.262.jar
│
├── output/
│
├── spark_jobs/
│   └── postgres_to_minio.py
│
├── docker-compose.yml
│
└── README.md
```

Step 1 — Install Required Software

Install Docker Desktop

Download Docker Desktop:

https://www.docker.com/products/docker-desktop/

Verify installation:

```
docker --version
```

Step 2 — Create Docker Compose File

Create `docker-compose.yml`

```
services:

  postgres:
    image: postgres:15
    container_name: postgres_db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: sales_db
    ports:
      - "5433:5432"

  spark:
    image: apache/spark:3.5.1
    container_name: spark_container
    command: >
      /opt/spark/bin/spark-class
      org.apache.spark.deploy.master.Master
    ports:
      - "8080:8080"
      - "7077:7077"
    volumes:
      - ./spark_jobs:/opt/spark_jobs
      - ./jars:/opt/jars
      - ./extra_jars:/opt/extra_jars
      - ./output:/opt/output

  minio:
    image: minio/minio
    container_name: minio_container
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    ports:
      - "9000:9000"
      - "9001:9001"
```

```
  airflow:
    image: apache/airflow:2.9.1
    container_name: airflow_container
    ports:
      - "8081:8080"
    command: standalone
    volumes:
      - ./airflow/dags:/opt/airflow/dags
      - ./spark_jobs:/opt/spark_jobs
      - ./jars:/opt/jars
      - ./extra_jars:/opt/extra_jars
      - /var/run/docker.sock:/var/run/docker.sock
```

Step 3 — Start All Containers

Run:

```
docker compose up -d
```

Verify containers:

```
docker ps
```

Expected containers:

- postgres_db
- spark_container
- minio_container
- airflow_container

Step 4 — Setup PostgreSQL

Connect Using pgAdmin

Connection details:

Parameter	Value
Host	localhost
Port	5433
Username	admin
Password	admin123
Database	sales_db

Create Orders Table

```
CREATE TABLE orders (
    order_id INT,
    customer_name VARCHAR(100),
    amount INT
);
```

Insert Sample Data

```
INSERT INTO orders VALUES
(1, 'samir', 1000),
(2, 'rahul', 2000),
(3, 'amit', 3000);
```

Step 5 — Download Required JAR Files

PostgreSQL JDBC Driver

Download:

https://jdbc.postgresql.org/download/

Place inside:

```
jars/
```

Example:

```
postgresql-42.7.11.jar
```

Hadoop AWS JAR

Download:

https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.3.4/hadoop-aws-3.3.4.jar

---

AWS SDK Bundle

Download:

https://repo1.maven.org/maven2/com/amazonaws/aws-java-sdk-bundle/1.12.262/aws-java-sdk-bundle-1.12.262.jar

Place both inside:

```
extra_jars/
```

Step 6 — Create Spark ETL Job

Create file:

```
spark_jobs/postgres_to_minio.py
```

Code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import upper

spark = SparkSession.builder
    .appName("PostgresToMinIO")
    .getOrCreate()

# MinIO Configuration
```

```
hadoop_conf = spark._jsc.hadoopConfiguration()

hadoop_conf.set("fs.s3a.access.key", "admin")
hadoop_conf.set("fs.s3a.secret.key", "password123")
hadoop_conf.set("fs.s3a.endpoint", "http://host.docker.internal:9000")
hadoop_conf.set("fs.s3a.path.style.access", "true")
hadoop_conf.set("fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")

jdbc_url = "jdbc:postgresql://host.docker.internal:5433/sales_db"

properties = {
    "user": "admin",
    "password": "admin123",
    "driver": "org.postgresql.Driver"
}

# Read Data From PostgreSQL
orders_df = spark.read.jdbc(
    url=jdbc_url,
    table="orders",
    properties=properties
)

print("Original Data")
orders_df.show()

# Transformation
transformed_df = orders_df.withColumn(
    "customer_name",
    upper(orders_df.customer_name)
)

print("Transformed Data")
transformed_df.show()

# Write To MinIO
transformed_df.write.mode("overwrite").parquet(
    "s3a://sales-data/orders_parquet"
)

spark.stop()
```

Step 7 — Setup MinIO Bucket

Open MinIO UI:

```
http://localhost:9001
```

Login credentials:

Username	Password
admin	password123

Create bucket:

```
sales-data
```

Step 8 — Run Spark Job Manually

Run:

```
docker exec -it spark_container /opt/spark/bin/spark-submit --jars /opt/jars/
postgresql-42.7.11.jar,/opt/extra_jars/hadoop-aws-3.3.4.jar,/opt/extra_jars/aws-
java-sdk-bundle-1.12.262.jar /opt/spark_jobs/postgres_to_minio.py
```

Step 9 — Verify Output In MinIO

Open bucket:

```
sales-data/orders_parquet
```

Expected files:

```
_SUCCESS
part-0000....parquet
```

Step 10 — Setup Airflow Credentials

Create Airflow admin user:

```
docker exec -it airflow_container airflow users create --username admin --
firstname Admin --lastname User --role Admin --email admin@example.com --
password admin123
```

Step 11 — Install PySpark Inside Airflow Container

```
docker exec -it --user airflow airflow_container python -m pip install pyspark
```

Step 12 — Create Airflow DAG

Create file:

```
airflow/dags/postgres_to_minio_dag.py
```

Code:

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

default_args = {
    "owner": "samir",
    "start_date": datetime(2026, 5, 28)
}

dag = DAG(
    dag_id="postgres_to_minio_pipeline",
    default_args=default_args,
    schedule=None,
    catchup=False
)

spark_job = BashOperator(
    task_id="run_spark_job",
    bash_command="""
docker exec spark_container
/opt/spark/bin/spark-submit
--jars /opt/jars/postgresql-42.7.11.jar,/opt/extra_jars/hadoop-aws-3.3.4.jar,/
```

```
opt/extra_jars/aws-java-sdk-bundle-1.12.262.jar
/opt/spark_jobs/postgres_to_minio.py
""",
    dag=dag
)

spark_job
```

Step 13 — Open Airflow UI

Open:

```
http://localhost:8081
```

Credentials:

Username	Password
admin	admin123

Step 14 — Trigger Airflow DAG

1. Open DAG
2. Enable DAG
3. Click Trigger DAG

Expected task flow:

```
queued → running → success
```

Final Working Flow

```
Airflow
   ↓
Spark Job
   ↓
Read PostgreSQL Data
```

```
   ↓
Transform Data
   ↓
Write Parquet To MinIO
```

Important Concepts Learned

1. Docker Compose

Used for running multiple containers together.

---

2. PostgreSQL

Used as OLTP source database.

---

3. Spark

Used for distributed data processing and transformation.

---

4. Parquet

Columnar storage format optimized for analytics.

---

5. MinIO

Local S3-compatible object storage.

---

6. Airflow

Used for workflow orchestration and scheduling.

Real Industry Equivalent

Local Setup	Cloud Equivalent
PostgreSQL	RDS
Spark	EMR / Databricks
MinIO	AWS S3
Airflow	MWAA / Composer
Docker	Kubernetes / ECS

Common Troubleshooting

Spark Driver Not Found

Issue:

```
ClassNotFoundException: org.postgresql.Driver
```

Solution:

Pass JDBC jar in spark-submit.

JAVA_HOME Error

Issue:

```
JAVA_HOME is not set
```

Solution:

Run Spark inside Spark container.

Volume Not Visible

Solution:

```
docker compose up -d --force-recreate
```

Airflow DAG Not Visible

Wait 30-60 seconds and refresh UI.

Future Improvements

Beginner Level

- Incremental Load
- Logging
- Config File
- Partitioned Parquet

Intermediate Level

- CDC Pipeline
- Data Quality Checks
- Scheduling
- Email Alerts

Advanced Level

- Kafka Streaming
- Snowflake Integration
- dbt
- CI/CD
- Kubernetes

Resume Project Description

Project Name

Containerized ETL Pipeline Using Airflow, Spark, PostgreSQL and MinIO

Resume Description

Designed and implemented an end-to-end containerized ETL pipeline using Apache Spark, PostgreSQL, MinIO and Apache Airflow. Built Docker-based orchestration for extracting data from PostgreSQL, applying Spark transformations, and storing parquet files in S3-compatible object storage. Automated workflow execution using Airflow DAGs.

Commands Cheat Sheet

Start Containers

```
docker compose up -d
```

Stop Containers

```
docker compose stop
```

Restart Containers

```
docker compose restart
```

Remove Containers

```
docker compose down
```

View Running Containers

```
docker ps
```

View Airflow Logs

```
docker logs airflow_container
```

Trigger Spark Job Manually

```
docker exec -it spark_container /opt/spark/bin/spark-submit --jars /opt/jars/
postgresql-42.7.11.jar,/opt/extra_jars/hadoop-aws-3.3.4.jar,/opt/extra_jars/aws-
java-sdk-bundle-1.12.262.jar /opt/spark_jobs/postgres_to_minio.py
```

Conclusion

This project demonstrates a complete modern Data Engineering workflow including:

- Source ingestion
- Distributed data processing
- Object storage
- Workflow orchestration
- Containerized deployment

The architecture closely resembles real-world cloud data engineering platforms used in production systems.